

Digital System Design

Dr. Nazanin Ghasemian

Fall 2026 · Lecture 5



Last Lecture

- Reviewed Sequential Logic
- Reviewed some common sequential logic elements
- Talk more about System Verilog

What Is a State?

- A state represents everything the circuit needs to remember about the past to determine its future behavior.
- For example, consider a digital lock that opens when it receives the sequence 101.
- The states could represent:

S0

No correct bits have been received

S1

The last received bit was 1

S2

The last two received bits were 10

S3

The complete sequence 101 has been detected

The state does not store the entire input history — only the information relevant to the circuit's future behavior.

A State Machine Models Sequential Behavior

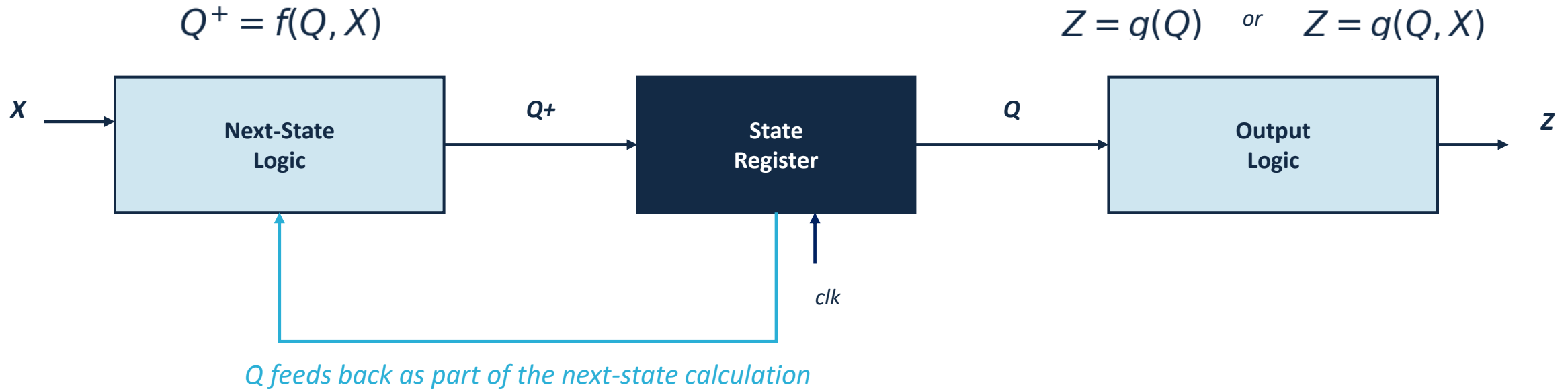
- A finite state machine, or FSM, is a model that shows:
 - The possible states of a circuit
 - The conditions that cause transitions between states
 - The output generated in each state or during each transition
- At each active clock edge:



Finite means the circuit has a limited number of possible states.

The Three Parts of a State Machine

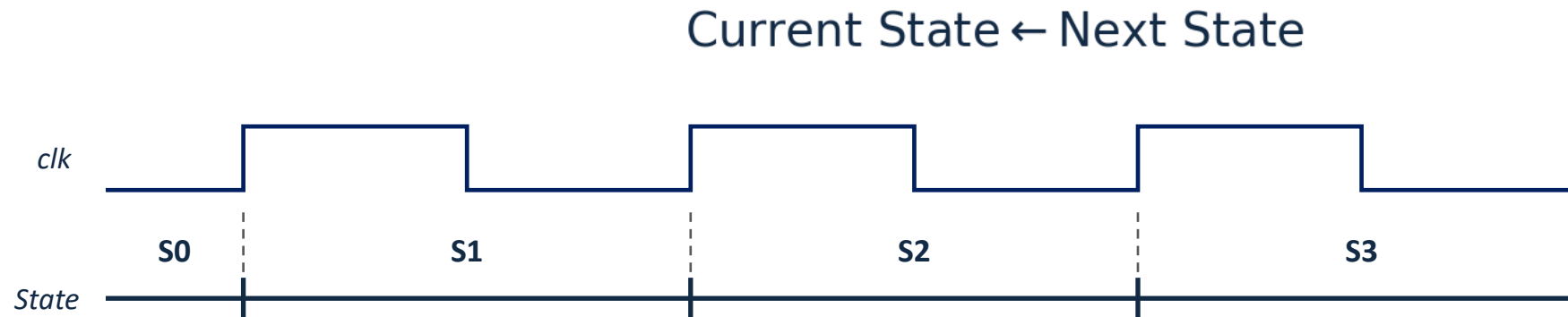
A synchronous state machine normally contains three major components:



Q = current state Q^+ = next state X = input Z = output

State Transitions Occur at the Clock Edge

- In a synchronous FSM, state transitions happen only at the active edge of the clock signal.
- Between active clock edges, the state stays fixed — even if the input changes.
- At the active edge, the state register loads the next state computed by the next-state logic:



The state only changes in step with the clock — never in between edges.

State Diagrams Visualize an FSM

A state diagram is a graphical representation of a finite state machine.

- Circles represent the states of the circuit.
- Arrows represent transitions between states.
- Each arrow is labeled with the input value (and output, for a Mealy machine) that causes the transition.



Example: if the circuit is in S0 and the input is 1, it moves to S1; on a 0, it stays in S0.

The State Transition Table

- A state transition table lists, for every combination of current state and input, the next state (and output) the circuit produces.
- It contains exactly the same information as the state diagram, in tabular form.
- Transition tables are especially useful for deriving the next-state logic equations.

Transition table for the
2-state example above

Current State	Input	Next State
S0	0	S0
S0	1	S1
S1	0	S0
S1	1	S1

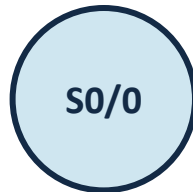
Each row of the table corresponds to exactly one arrow in the state diagram.

Two Types of Finite State Machines

FSMs are classified by how they generate their output:

Moore Machine

- Output depends only on the current state
- Written as State / Output
- Output changes only at clock edges



Mealy Machine

- Output depends on the current state and the current input
- Written on the arrow as Input / Output
- Output can change as soon as the input changes

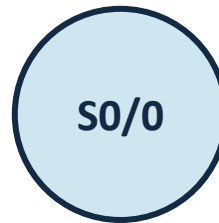


The Moore Machine

- In a Moore machine, the output is a function of the current state only:

$$Z = q(Q)$$

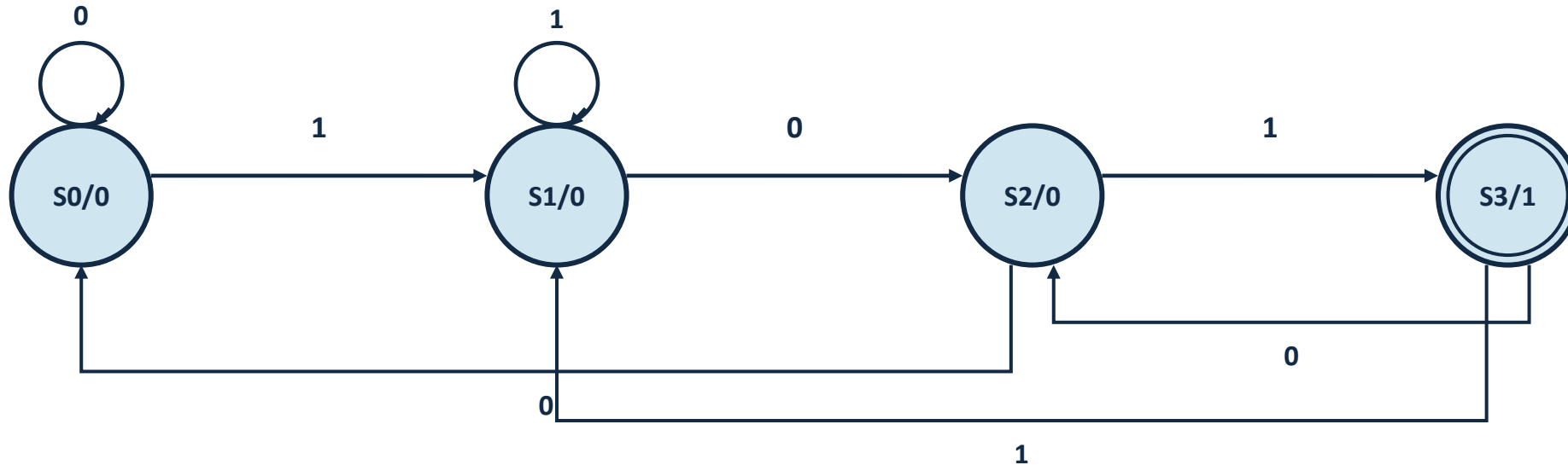
- Each state is labeled with the output it produces, written as State / Output.
- Because the output does not depend on the input, it is stable and glitch-free within a state.
- The output can only change when the state changes — that is, at a clock edge.



State S0 always outputs 0, for as long as the circuit remains in S0.

Moore Machine Example: Detecting 101

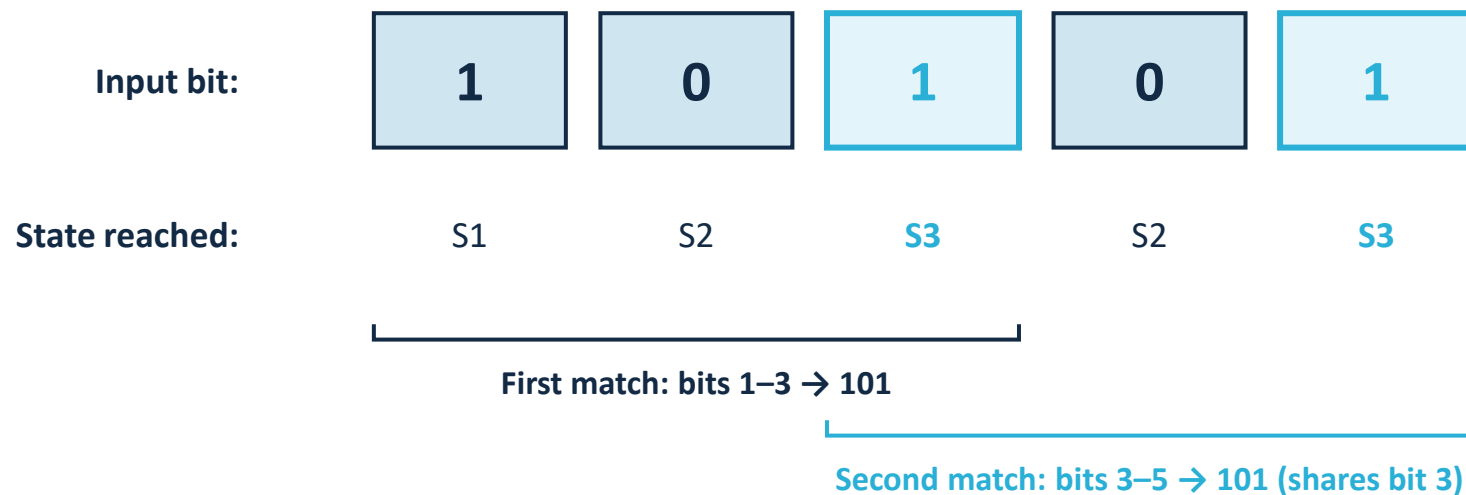
This Moore machine detects the overlapping sequence 101 in a serial input stream, using the states defined earlier.



Current State	Input = 0	Input = 1	Output
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S2	S1	1

Understanding Overlapping Sequences

- After detecting 101, state S3 does not always return to S0 — it moves to whatever state matches the most recent bits.
- This lets the final 1 of one detected sequence also serve as the first 1 of the next — so overlapping occurrences of 101 are not missed.



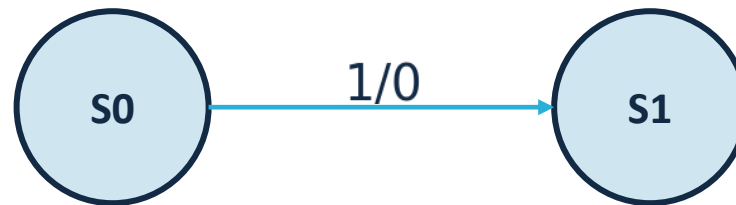
Bit 3 is used in both matches — this is only possible because overlaps are allowed.

The Mealy Machine

- In a Mealy machine, the output is a function of both the current state and the current input:

$$Z = q(Q, X)$$

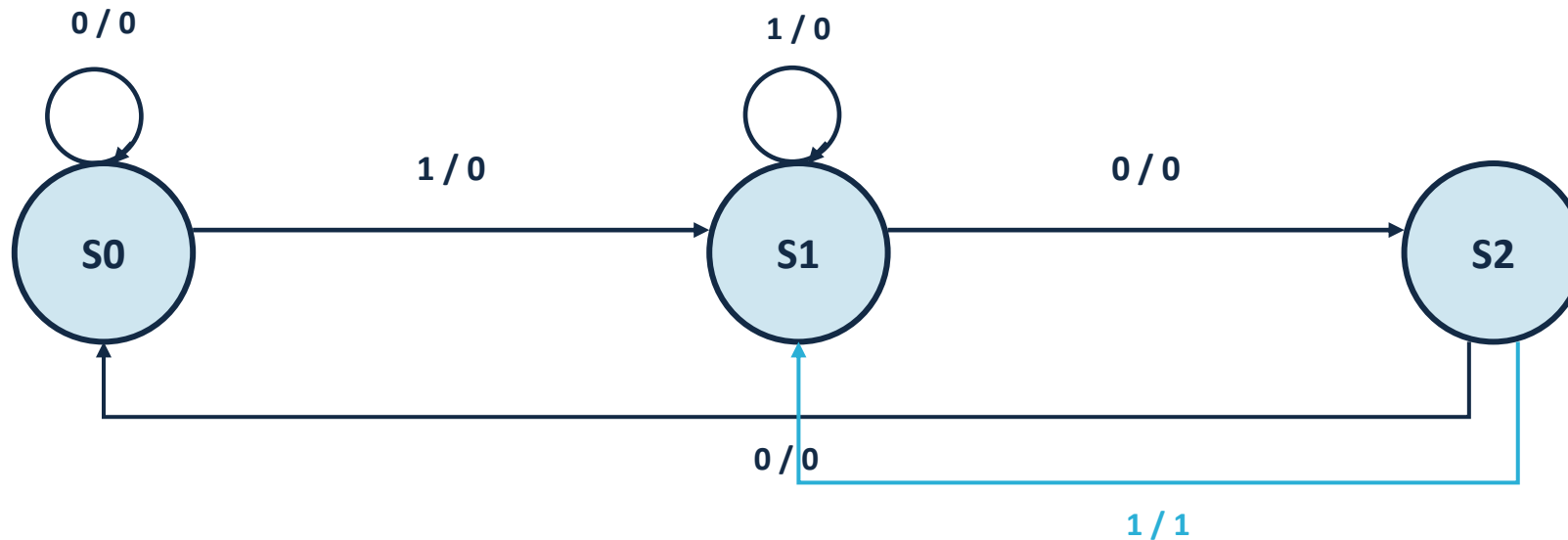
- The output is written on the transition arrow, as Input / Output, rather than inside the state.
- Because the output depends on the input directly, it can change as soon as the input changes — even between clock edges.
- A Mealy machine often needs fewer states than an equivalent Moore machine.



Reaching S1 outputs a 1 immediately when the input is 1 — no clock edge required.

Mealy Machine Example: Detecting 101

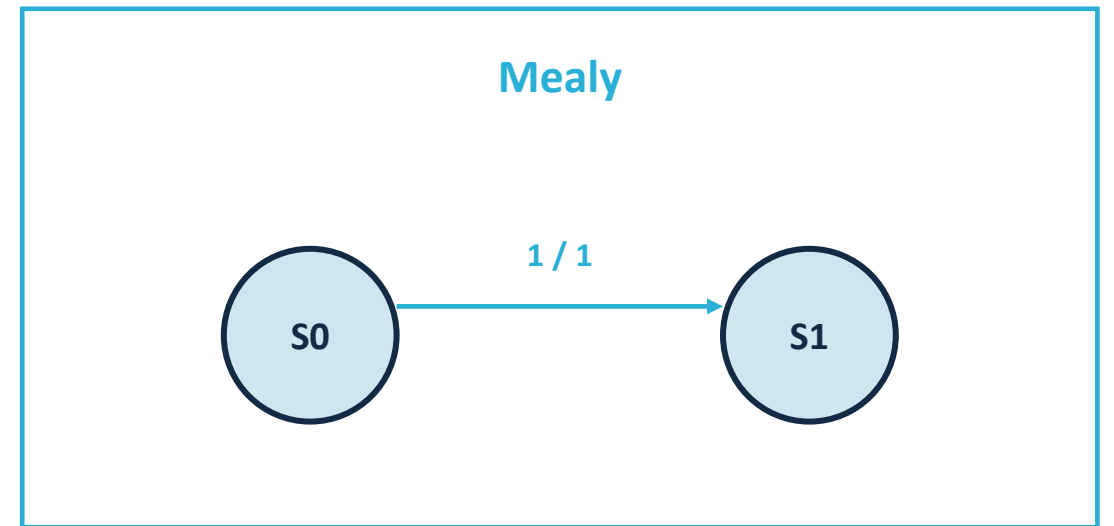
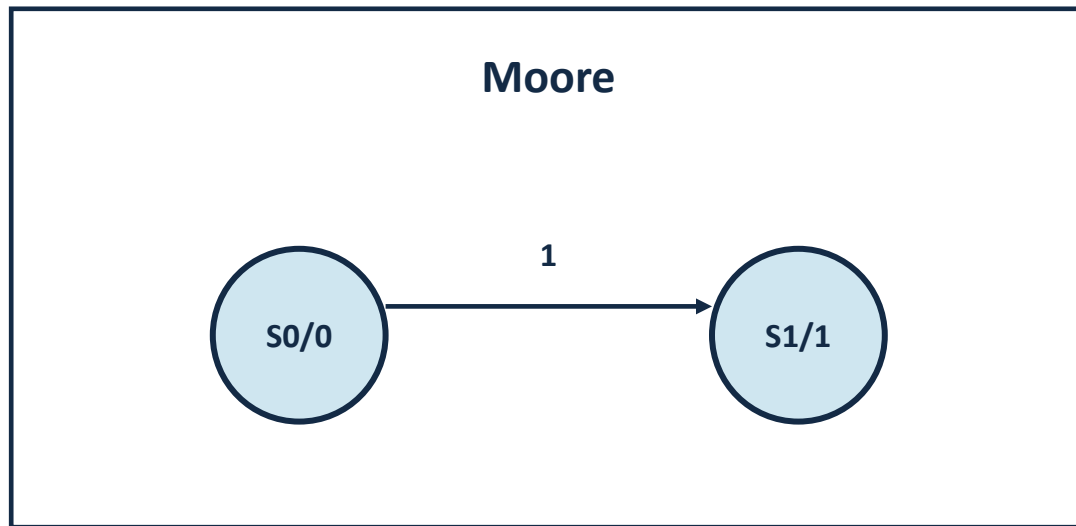
This same 101 detector needs only 3 states as a Mealy machine — S3 becomes an output on the S2 → S1 arrow.



Current State	Input = 0	Input = 1
S0	S0 / 0	S1 / 0
S1	S2 / 0	S1 / 0
S2	S0 / 0	S1 / 1

Moore and Mealy Diagrams Use Different Labels

You can tell a Moore diagram from a Mealy diagram just by where the output appears.



How to tell them apart:

Output written inside a state (State / Output) → Moore.

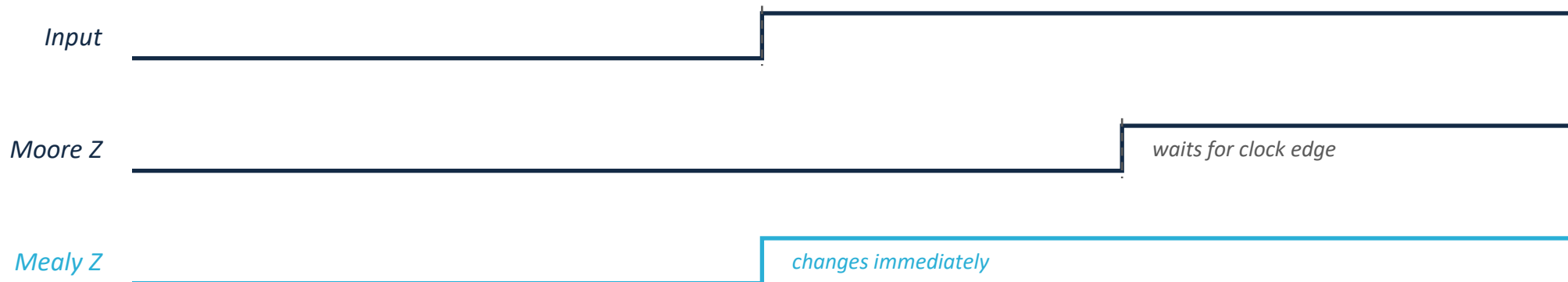
Output written on an arrow (Input / Output) → Mealy.

Moore vs. Mealy: Side-by-Side Comparison

Feature	Moore Machine	Mealy Machine
Output depends on	Current state only	Current state and current input
Output notation	State / Output	Input / Output (on the arrow)
Output timing	Changes only at clock edges	Can change as soon as the input changes
Typical state count	Often more states needed	Often fewer states needed
Output stability	Stable, glitch-free within a state	Can glitch if the input is noisy
Design complexity	Simpler to analyze and verify	Can be more compact but trickier to time

Why Mealy Outputs Can Change Earlier

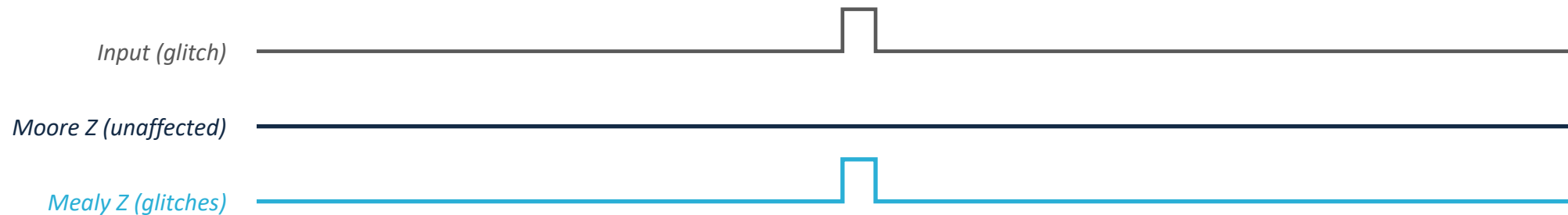
- A Moore output depends only on the state, so it can only change once the state register updates — at the next clock edge.
- A Mealy output depends directly on the input, so it can change the moment the input changes, without waiting for a clock edge.
- This means a Mealy machine can respond to an input one clock cycle sooner than an equivalent Moore machine.



This faster response is a key reason designers sometimes prefer Mealy machines.

A Practical Caution About Mealy Outputs

- Because a Mealy output responds directly to the input, it can also react to unwanted, momentary input changes.
- If the input is noisy or glitchy, the output can glitch too — even though the state has not changed.
- A Moore output cannot do this: since it depends only on the (registered) state, it is inherently glitch-free.
- In practice, Mealy outputs are usually sampled or registered before being used, to avoid propagating input glitches downstream.



Rule of thumb: choose Moore for stable outputs; choose Mealy when a faster response outweighs the risk of glitches.

State Encoding

- Each abstract state (S0, S1, S2, ...) must be assigned an actual binary code before the circuit can be built.
- For N states, the state register needs at least this many bits:

$$\lceil \log_2 N \rceil$$

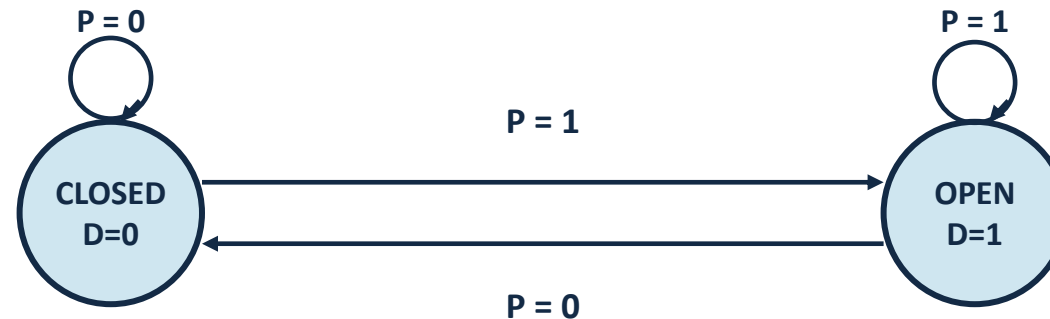
- Common encoding styles:

Encoding	How it works	Example (4 states)
Binary	States numbered in standard binary	S0=00, S1=01, S2=10, S3=11
One-hot	One flip-flop per state; exactly one bit is 1	S0=0001, S1=0010, S2=0100, S3=1000
Gray code	Adjacent states differ by exactly one bit	S0=00, S1=01, S2=11, S3=10

One-hot uses more flip-flops but often simpler, faster next-state logic; binary minimizes flip-flops.

Example Design Question: Automatic Door

- Design an FSM for an automatic door with a presence sensor input P and a motor-drive output D.
- $P = 1$ while someone is detected in front of the door; $D = 1$ drives the door open.



Questions to consider:

- Is this a Moore or a Mealy machine, as drawn?
- What happens if P briefly flickers while the door is CLOSED?
- Would you add a timer or additional states for a more realistic door?

Common FSM Design Mistakes

- Forgetting to define behavior for every input in every state
- Confusing Moore outputs (in the state) with Mealy outputs (on the arrow)
- Using too few states, so the FSM cannot distinguish important histories
- Using too many states, making the design unnecessarily complex
- Forgetting that transitions happen only at the active clock edge
- Mixing encodings inconsistently between the diagram, table, and logic
- Ignoring metastability or glitch risk on Mealy outputs
- Not verifying the design against every scenario in the specification
- Skipping the state transition table and jumping straight to logic

Key Takeaways

- A sequential circuit's output depends on the current input and its current state — its memory of the past.
- A finite state machine is built from a state register, next-state logic, and output logic.
- State transitions happen only at active clock edges, keeping the whole system synchronized.
- Moore machines produce stable, glitch-free outputs tied to the state; Mealy machines can respond faster but depend directly on the input.
- A disciplined design procedure — states, diagram, table, encoding, logic — keeps FSM designs correct and easy to verify.

Sequential circuits are the foundation of every counter, controller, and protocol engine you will build next.